

Agentic Autoresearch for Discrete Text Diffusion Transformers on Consumer Hardware

A Controlled Comparison of Uniform (SEDD) and Masked (MDLM) Objectives

Goutham Krishnan
goutham.krishnan22@gmail.com

Abstract—Discrete diffusion has emerged as a non-autoregressive alternative to left-to-right language modeling, but most reported results depend on large compute budgets that place careful architectural exploration out of reach for practitioners on consumer hardware. We present TON-v1, a 77.2M-parameter bidirectional diffusion transformer trained to generate short children’s stories on a single 8GB RTX 4060 laptop GPU. Our central methodological contribution is *agentic autoresearch*: an autonomous large-language-model agent iteratively edits the training script, trains under a fixed wall-clock budget, reads back a target metric, and keeps an edit only if the metric improves. Using this loop we conduct two controlled studies on an identical transformer backbone. The first tunes a Score Entropy Discrete Diffusion (SEDD, uniform) model over 52 experiments, reducing validation loss by 39%, with a single change, replacing learned absolute position embeddings with rotary embeddings (RoPE), accounting for a 28% reduction on its own. The second replaces the diffusion objective with a Masked Diffusion Language Model (MDLM, absorbing) formulation and, guided by a loss-agnostic generative-perplexity metric, reduces generative perplexity from 350.5 to 42.6 over 12 experiments (an $8.2\times$ improvement). Full training of the best MDLM configuration for 100,000 steps reaches a generative perplexity of 24.75 with a distinct-2 diversity of 0.68, producing fluent, non-degenerate text. We report full ablation logs, generation-time latency and memory benchmarks, and a candid account of which interventions helped and which did not, as a reproducible baseline for small-scale discrete diffusion research.

Index Terms—discrete diffusion, masked diffusion, text generation, transformers, non-autoregressive generation, automated machine learning, rotary position embeddings, consumer GPU

I. INTRODUCTION

Autoregressive (AR) language models generate text one token at a time, conditioning each token only on its predecessors. Discrete diffusion models offer an alternative: they generate an entire sequence in parallel by iteratively denoising a corrupted sequence, allowing every position to attend bidirectionally to every other at every step. Recent formulations, Score Entropy Discrete Diffusion (SEDD) [4] and Masked Diffusion Language Models (MDLM) [5], have narrowed the perplexity gap to AR models, but the design decisions behind these results are typically established at a scale unavailable to most practitioners.

This paper asks a practical question: *how far can a small discrete diffusion transformer be pushed on a single consumer*

GPU, and how much of the tuning can be delegated to an autonomous agent? We make three contributions:

- 1) **An agentic autoresearch methodology** in which an LLM agent autonomously proposes, implements, runs, and accepts-or-rejects training-script edits under a fixed wall-clock budget, using an objective metric as the sole acceptance criterion.
- 2) **A controlled comparison** of the uniform (SEDD) and masked/absorbing (MDLM) diffusion objectives on an identical 77.2M-parameter backbone, isolating the effect of the corruption process and loss from confounding architectural differences.
- 3) **A fully reproducible baseline**, including complete per-experiment ablation logs, generation-quality metrics, and inference-time latency/memory benchmarks, all obtained on an 8 GB RTX 4060 laptop GPU.

II. BACKGROUND AND RELATED WORK

A. Discrete Diffusion

Continuous diffusion models [1], [2] corrupt data with Gaussian noise and learn to reverse the process. For discrete data such as text, the corruption is instead a Markov chain over token states [3]. Two corruption kernels are relevant here. In the *uniform* kernel, a token is replaced by another token drawn uniformly from the vocabulary; SEDD [4] learns the concrete score (the ratio of marginal probabilities between states) and optimizes a denoising score entropy objective. In the *absorbing* kernel, a token is replaced by a dedicated [MASK] state that is never reversed except by prediction; MDLM [5] shows that this reduces to a simple, time-weighted masked cross-entropy objective, essentially a diffusion-theoretic generalization of masked language modeling [7], and reports improved perplexity over SEDD.

B. Architectural Ingredients

Our backbone is a standard transformer encoder [6] with bidirectional attention. We adopt rotary position embeddings (RoPE) [8] for relative positional encoding, adaptive layer normalization (adaLN) [9] to condition every block on the diffusion noise level, and query-key normalization for attention

TABLE I
SHARED BACKBONE AND TRAINING CONFIGURATION

Component	Value
Parameters	$\approx 77.2\text{M}$
Layers / width / heads	6 / 512 / 16
Context length	128 tokens
Attention	bidirectional, RoPE (base 1000), QK-norm
Time conditioning	sinusoidal $\rightarrow$ MLP $\rightarrow$ adaLN per block
Tokenizer / vocabulary	GPT-2 BPE / 50,257 (+1 for MDLM)
Precision	bf16 autocast, TF32 matmuls
Optimizer	fused Adam, grad-clip 1.0
Batch size	16
LR schedule	100-step warmup, cosine decay
Peak LR (SEDD / MDLM)	1×10^{-3} / 3×10^{-3}

stability. The vocabulary and tokenizer are inherited from GPT-2 [10].

C. Automated and Agentic Model Search

Classical neural architecture search automates discrete design choices but is compute-intensive. Recent work explores using LLM agents as autonomous researchers that write and evaluate their own code. Our loop follows the spirit of Karpathy’s “autoresearch” formulation [12]: a tight edit-train-measure-decide cycle on a single script under a fixed budget. To our knowledge this is among the first documented applications of such a loop to discrete diffusion language models on consumer hardware.

III. MODEL ARCHITECTURE

Both studies share one backbone (Table I). A sequence of $T=128$ token embeddings of width $d=512$ passes through six identical pre-norm transformer blocks. Each block applies (i) multi-head self-attention with 16 heads, RoPE, and RMSNorm applied to per-head queries and keys, followed by (ii) a position-wise feed-forward network of hidden width $4d$. Attention is computed with a fused scaled-dot-product kernel and is fully bidirectional. The diffusion time $t \in [0, 1]$ is mapped through a sinusoidal embedding and an MLP to a conditioning vector that modulates each block via adaLN (a per-block scale and shift on the normalized activations).

The only architectural difference between the two models is the token embedding table. The MDLM variant adds one row for the absorbing [MASK] state, giving $|\mathcal{V}|+1$ input rows against a $|\mathcal{V}|$ -way output head; the SEDD variant uses $|\mathcal{V}|$ rows throughout. This accounts for a difference of exactly $d=512$ parameters (77,236,689 for SEDD versus 77,237,201 for MDLM), i.e. both models are $\approx 77.2\text{M}$ parameters.

IV. TRAINING OBJECTIVES

Let x_0 be a clean token sequence and $t \sim \mathcal{U}(0, 1)$ a sampled noise level (drawn with stratified sampling across the batch to reduce gradient variance).

A. SEDD (Uniform)

The forward process replaces tokens with uniform random tokens under a geometric noise schedule. The model outputs per-position log-ratios s_θ , trained with a denoising weighted score entropy (DWDSE) objective. We use a closed-form evaluation of the loss that avoids materializing the full rate and transition tensors, which reduced the forward pass from 72 ms to 35 ms per step in our implementation.

B. MDLM (Absorbing)

The forward process independently replaces each token with [MASK] with probability t (a linear schedule). The model predicts the original token at each masked position, and the negative ELBO reduces to a time-weighted cross-entropy over masked positions only:

$$\mathcal{L}_{\text{MDLM}} = \mathbb{E}_{t, x_0, x_t} \left[\frac{1}{t} \sum_{i: x_t^{(i)} = [\text{MASK}]} -\log p_\theta(x_0^{(i)} | x_t) \right]. \quad (1)$$

The $1/t$ weight upweights lightly-corrupted examples, whose correct reconstruction is a stronger learning signal. Generation runs the reverse process from an all-[MASK] sequence, progressively unmasking positions over N steps; at each step we sample predicted tokens with top- k filtering.

V. AGENTIC AUTORESEARCH METHODOLOGY

The tuning of both models was performed by an autonomous LLM agent, not by hand. The agent executes the following loop indefinitely until stopped:

- 1) Inspect the current git state and the training script.
- 2) Propose and directly implement one experimental change (any part of the architecture, optimizer, schedule, loss, or sampler is in scope).
- 3) Commit, then launch training under a fixed wall-clock budget (5 minutes for SEDD, 10 minutes for MDLM), redirecting all output to a log.
- 4) Parse the final target metric from the log.
- 5) If the metric improved, advance the branch (keep the commit); otherwise revert to the prior state. Log the outcome, with a one-line rationale, to a tab-separated results file.

Because the budget is wall-clock rather than a fixed step count, the loop implicitly rewards throughput: any change that increases steps-per-second lets the step-bound model train longer within the same budget, which is itself a source of metric improvement. This design made low-level efficiency work (mixed precision, fused kernels, closed-form losses) a first-class part of the search rather than an afterthought.

A. Choice of Target Metric

The two objectives are not numerically comparable: DWDSE and weighted cross-entropy live on different scales, and the raw SEDD loss values we report are unnormalized, implementation-specific quantities useful only for *relative* comparison within a single study. For the SEDD study we

therefore optimized held-out DWDSE directly. For the MDLM study, where the goal was to compare against and surpass the SEDD baseline, we introduced a loss-agnostic quality metric: **generative perplexity** of the model’s own samples under a frozen GPT-2 [10] scorer (valid because our samples are already GPT-2 tokens), guarded by a **distinct-2** bigram-diversity measure to detect degenerate low-perplexity output (e.g., repetition). All MDLM acceptance decisions used generative perplexity, with distinct-2 as a veto.

VI. EXPERIMENTAL SETUP

All experiments ran on a single NVIDIA RTX 4060 Laptop GPU (8GB) under Windows. Data is the `karpathy/tinystories-gpt4-clean` corpus [11], a set of simple short stories; the full corpus of $\approx 542\text{M}$ GPT-2 tokens was tokenized once and cached to disk, with a 90/10 train/validation split. The autoresearch loops used a token subset for speed; the final full-training runs used the entire corpus. Held-out evaluation uses a fixed random seed so that reported losses depend only on model weights and are comparable across commits within a study.

VII. RESULTS

A. SEDD Autoresearch

Over 52 experiments (22 accepted), the agent reduced held-out DWDSE from a baseline of 325,306 to 198,472, a 39% reduction (Table II). Early gains were dominated by throughput interventions: TF32 matmuls, fused QKV with scaled-dot-product attention, a training-only bf16 pass, and the closed-form DWDSE loss, each of which increased steps-per-second and thus effective training within the fixed budget. A sequence of small architectural wins followed: QK-norm, sinusoidal time conditioning, adaLN, 16 attention heads, and a reduction from 8 to 6 layers (the model is step-bound, not capacity-bound, so depth traded unfavorably against steps). The single largest intervention was replacing the learned absolute position embedding with RoPE, which reduced the loss by 28% in one step (279,226 $\rightarrow$ 201,269); tuning the RoPE base to 1000 for the short context yielded the best result of 198,472. The agent also correctly rejected numerous plausible-but-unhelpful ideas, including weight tying (which diverged, as the output head feeds an exponential and cannot share the input embedding), SwiGLU, wider embeddings, importance sampling of t , and both larger and smaller batch sizes.

B. MDLM Autoresearch

Starting from the tuned SEDD backbone, the agent replaced the objective with MDLM and, guided by generative perplexity, ran 12 experiments (7 accepted). Switching the corruption process and loss alone, with no architectural change, reduced generative perplexity from 350.5 to 221.7 in a single step while simultaneously improving diversity (distinct-2 0.70 $\rightarrow$ 0.85), confirming the reported advantage of the absorbing objective. Raising the peak learning rate to 3×10^{-3} , increasing the number of sampling steps to 256 (the single largest sampling win, 186 $\rightarrow$ 115), and top- $k=20$ sampling drove generative

TABLE II
SELECTED SEDD AUTORESEARCH MILESTONES (HELD-OUT DWDSE)

Intervention	Val loss	Note
Baseline	325,306	tuned starting point
TF32 matmul	290,336	throughput
Fused Adam	294,316	throughput
QK-norm	286,682	stability
Closed-form DWDSE	285,160	+27% steps/s
16 heads	284,253	finer attention
adaLN (scale+shift)	280,337	time conditioning
RoPE	201,269	single largest win
RoPE base 1000	198,472	best

TABLE III
MDLM AUTORESEARCH PROGRESSION (GENERATIVE PERPLEXITY)

Intervention	Gen. PPL	Distinct-2	Status
SEDD baseline	350.5	0.70	keep
$\rightarrow$ MDLM objective	221.7	0.85	keep
LR 3×10^{-3}	186.2	0.84	keep
256 sampling steps	115.1	0.76	keep
top- $k=50$	74.8	0.75	keep
top-$k=20$	42.6	0.66	keep
top- $k=10$	41.8	0.60	discard

perplexity to 42.6 (Table III). The distinct-2 guard proved its worth: top- $k=10$ matched the perplexity of top- $k=20$ but reduced diversity to 0.60 with visible repetition, and was correctly rejected.

C. Full Training

The best configuration of each model was then trained to convergence on the full corpus (not budget-limited). The SEDD model trained for 75,000 steps, reaching a best held-out DWDSE of 124,395. The MDLM model trained for 100,000 steps, reaching a final generative perplexity of **24.75** with distinct-2 of 0.68. For reference, real TinyStories text scores in the approximate 30 to 50 range under the same GPT-2 scorer; the trained MDLM model therefore produces samples whose fluency, as measured by this proxy, is comparable to the training distribution while retaining healthy lexical diversity. A representative uncensored sample:

“One day, the smart cat saw a big dog. The dog wanted [the] tree and the cat. The cat was in the tree... The cat and the bird became friends. They played and ate [by] the tree together. The cat was happy again.”

D. Inference Benchmarks

Table IV reports single-sequence generation latency and peak memory on the RTX 4060, averaged over 20 runs after warmup. Because generation is non-autoregressive, there is no key-value cache and no prompt-prefill asymmetry: every denoising step is a full-sequence bidirectional forward pass of near-identical cost. Total latency is thus dominated by the number of denoising steps. The MDLM model, tuned to 256 steps, is correspondingly slower per sample than the 128-step SEDD model, but both generate a full 128-token story in a few seconds within ≈ 1.4 GB of memory, well inside the 8 GB

TABLE IV
GENERATION BENCHMARKS ON RTX 4060 (1 SEQUENCE, 128 TOKENS)

Metric	SEDD	MDLM
Denoising steps	128	256
Per-step latency (ms)	18.05	16.76
Total latency (ms)	2311	4293
Peak memory (MB)	1356	1330
Cold-start (ms)	3082	5303

budget. We note that per-sample latency amortizes strongly under batching, since a single sequence underutilizes the GPU.

VIII. DISCUSSION

A. What Worked

Three themes dominated the useful interventions. First, in a step-bound regime, *throughput is quality*: mixed precision, fused kernels, and a closed-form loss all improved the target metric purely by enabling more optimization steps within the fixed budget. Second, *relative positional information matters disproportionately*: RoPE was the single largest win in the SEDD study. Third, for the absorbing objective, *sampling-time decisions rival training-time ones*: the number of denoising steps and the top- k threshold together moved generative perplexity more than any single training change after the objective switch itself.

B. What Did Not

The agent’s rejected experiments are as informative as its accepted ones. Increased depth, increased width, and larger batch sizes all lost to the step-bound baseline. Weight tying diverged for a structural reason specific to the score parameterization. Importance sampling of the noise level, SwiGLU, and RMSNorm produced no benefit at this scale. The consistency of the “6 layers, 512 width, 16 heads” optimum across many re-tests suggests these are genuine local optima for this data-and-budget regime rather than noise.

C. Limitations

This is a small-scale study on a narrow, simple corpus; the absolute quality is far from large diffusion or AR language models, and we make no scaling claims. Generative perplexity under a fixed scorer is a proxy that can be gamed by low-entropy text, which is precisely why the distinct-2 guard was necessary, and why we caution against reading the sub-30 figure as surpassing real text in any absolute sense. The wall-clock budget couples results to the specific hardware. Finally, the autoresearch loop performs greedy, single-metric hill-climbing and does not explore interactions between rejected changes; a more sophisticated search could recover value from combinations it discarded individually.

IX. CONCLUSION

We demonstrated that an autonomous LLM agent can meaningfully tune a discrete diffusion transformer on a single consumer GPU, and used that loop to run a controlled comparison of the uniform (SEDD) and masked (MDLM) diffusion

objectives on an identical 77.2M-parameter backbone. The masked/absorbing objective, combined with agent-discovered sampling-time tuning, reduced generative perplexity by more than $8\times$ over the tuned uniform baseline and yielded fluent, non-degenerate short stories at a final generative perplexity of 24.75. We release the complete ablation logs, metrics, and benchmarks as a reproducible baseline for small-scale discrete diffusion research, and suggest agentic autoresearch as a practical tool for architecture and objective exploration under tight compute budgets.

REPRODUCIBILITY

All code, per-experiment logs (`results.tsv`, `results_v2.tsv`), generated samples, loss curves, and the benchmark harnesses used to produce every number in this paper are organized into two self-contained experiment directories, `diffusion_sedd/` and `diffusion_mdml/`.

REFERENCES

- [1] J. Sohl-Dickstein, E. Weiss, N. Maheswaranathan, and S. Ganguli, “Deep unsupervised learning using nonequilibrium thermodynamics,” in *Proc. ICML*, 2015.
- [2] J. Ho, A. Jain, and P. Abbeel, “Denoising diffusion probabilistic models,” in *Proc. NeurIPS*, 2020.
- [3] J. Austin, D. Johnson, J. Ho, D. Tarlow, and R. van den Berg, “Structured denoising diffusion models in discrete state-spaces,” in *Proc. NeurIPS*, 2021.
- [4] A. Lou, C. Meng, and S. Ermon, “Discrete diffusion modeling by estimating the ratios of the data distribution,” in *Proc. ICML*, 2024. arXiv:2310.16834.
- [5] S. Sahoo *et al.*, “Simple and effective masked diffusion language models,” in *Proc. NeurIPS*, 2024. arXiv:2406.07524.
- [6] A. Vaswani *et al.*, “Attention is all you need,” in *Proc. NeurIPS*, 2017.
- [7] J. Devlin, M. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding,” in *Proc. NAACL*, 2019.
- [8] J. Su, Y. Lu, S. Pan, B. Wen, and Y. Liu, “RoFormer: Enhanced transformer with rotary position embedding,” arXiv:2104.09864, 2021.
- [9] W. Peebles and S. Xie, “Scalable diffusion models with transformers,” in *Proc. ICCV*, 2023.
- [10] A. Radford *et al.*, “Language models are unsupervised multitask learners,” OpenAI Tech. Rep., 2019.
- [11] R. Eldan and Y. Li, “TinyStories: How small can language models be and still speak coherent English?” arXiv:2305.07759, 2023.
- [12] A. Karpathy, “autoresearch,” GitHub repository. [Online]. Available: <https://github.com/karpathy/autoresearch>